Звіт з лабораторної роботи №2

Середовище виконання: WSL

Для проведення профілювання та порівняльного аналізу з вихідного коду code.cpp було скомпільовано дві версії програми:

1. Базова версія (bubble_base):

g++ -O0 -g code.cpp -o app

Параметри команди:

- g++ - компілятор мови C++ із пакету GNU Compiler Collection (GCC), який здійснює трансляцію вихідного коду в машинні інструкції.
- -O0 - вимикає оптимізації компілятора (рівень оптимізації 0). Це дає змогу отримати базову версію програми без оптимізацій, що корисно для початкового аналізу продуктивності.
- -g - додає до виконуваного файлу налагоджувальну інформацію (debug symbols). Вона допомагає інструментам профілювання, зокрема perf і FlameGraph, відображати назви функцій та пов'язувати їх із рядками вихідного коду.
- code.cpp - вхідний файл із вихідним кодом програми мовою C++, який передається компілятору для обробки.
- -o bubble_base - задає назву вихідного виконуваного файлу (параметр -o означає output, a bubble_base - ім'я файлу, який буде створено в результаті компіляції).

2. Оптимізована версія (bubble_opt):

g++ -O3 -march=native -flto -g bubble_sorting.cpp -o bubble_opt

Параметри команди:

- g++ - компілятор мови C++ із пакету GNU Compiler Collection (GCC), який здійснює трансляцію вихідного коду в машинні інструкції.
- -O3 - високий рівень оптимізації компілятора, який вмикає агресивні перетворення коду задля досягнення максимальної продуктивності виконання програми.
- -march=native - оптимізує згенерований код спеціально під архітектуру процесора поточного комп'ютера, задіюючи всі доступні на ньому інструкції (наприклад, AVX).
- -flto - Link-Time Optimization (оптимізація на етапі компонування), яка дозволяє проводити загальну оптимізацію всього проєкту під час зв'язування модулів.

- -g - додає до виконуваного файлу налагоджувальну інформацію (debug symbols) для зручного профілювання за допомогою perf і FlameGraph.
- bubble_sorting.cpp - вхідний файл із вихідним кодом програми мовою C++ (реалізація сортування бульбашкою).
- -o bubble_opt - задає назву вихідного виконуваного файлу (-o - output, bubble_opt - ім'я оптимізованого бінарника).

Побудова FlameGraph

Перед аналізом роботи програми було налаштовано доступ до подій продуктивності Linux. Для цього виконується команда: sudo sysctl -w kernel.perf_event_paranoid=-1
Вона змінює параметр ядра Linux, який визначає обмеження доступу до подій продуктивності. Це може бути необхідним у середовищі WSL2, залежно від його конфігурації та прав доступу.

Для збору даних про виконання програми використовується команда:

sudo perf record -F 99 -g -- ./app

Параметри команди:

- perf record — запускає програму та записує дані профілювання.
- -F 99 — задає частоту семплювання приблизно 99 разів на секунду.
- -g — вмикає запис стеків викликів, що дає змогу відстежувати ланцюжки викликаних функцій.
- -- — відокремлює параметри perf від команди запуску програми.
- ./app — виконуваний файл, який профілюється.

Під час виконання програми perf періодично фіксує, де саме перебуває процесор, і записує інформацію, потрібну для подальшого аналізу. У даній лабораторній роботі результати зберігаються у файлі perf_base.data (perf_opt.data)

Дані, зібрані perf, потрібно перетворити у формат, який підтримує FlameGraph. Для цього використовується команда:

sudo perf script | ~/FlameGraph/stackcollapse-perf.pl > out.folded

Команда складається з двох основних етапів. Спочатку perf script перетворює записані дані профілювання на текстове представлення, що містить інформацію про семпли та стеки викликів. Потім скрипт stackcollapse-perf.pl об'єднує відповідні стеки викликів і підраховує семпли для кожного з них. Результат записується у файл out.folded. У даній лабораторній роботі результати команди зберігаються у файлі base.folded (opt.folded)

Цей файл містить агреговані дані, на основі яких будується Flame Graph.

На наступному етапі створюється графічне представлення результатів:

~/FlameGraph/flamegraph.pl out.folded > flamegraph_before.svg

Скрипт flamegraph.pl обробляє файл obase.folded і генерує інтерактивну SVG-візуалізацію. Отриманий граф зберігається у файлі flamegraph_before.svg, який можна відкрити у веббраузері.

Аналіз Flame Graph

За результатами профілювання базової версії програми bubble_base, скомпільованої без оптимізацій (-O0), можна зробити такі висновки.

Горизонтальна ширина блоків Flame Graph відображає частку зібраних семплів, пов'язаних із відповідними функціями. На графі видно, що основна частина процесорного часу припадає на функцію bubble_sort. Це пояснюється особливостями алгоритму сортування бульбашкою, часова складність якого в середньому та найгіршому випадках становить (O(N^2)). Для масиву розміром (N = 30000) алгоритм виконує значну кількість порівнянь і перестановок елементів.

Стек викликів на Flame Graph читається знизу вгору. У нижній частині відображаються функції, пов'язані із запуском програми, зокрема _start і __libc_start_main. Далі виконується функція main, з якої викликається bubble_sort.

Саме блок bubble_sort займає найбільшу частину графа, що підтверджує значні витрати процесорного часу на виконання сортування порівняно з іншими операціями програми.

На графі також помітний блок std::vector<int, std::allocator<int>>::operator[], пов'язаний із доступом до елементів вектора за індексом.
Оскільки програму скомпільовано з параметром -O0, компілятор не виконує звичайних оптимізацій, зокрема може не застосовувати інлайнінг функцій. Через це операції індексації вектора, наприклад arr[j] і arr[j + 1], можуть створювати додаткові накладні витрати.
Також на графі можуть бути помітні блоки, пов'язані зі створенням і ініціалізацією вектора та виділенням пам'яті. Якщо їхня ширина невелика порівняно з блоком bubble_sort, це свідчить про меншу частку відповідних операцій у загальній кількості зібраних семплів.

Профілювання базової версії програми показує, що основним джерелом обчислювальних витрат є функція bubble_sort. Значний обсяг операцій сортування та відсутність оптимізацій компілятора призводять до збільшення часу виконання. Отримані результати дають змогу надалі порівняти базову й оптимізовану версії програми та оцінити вплив параметрів компіляції на продуктивність.

Вимірювання часу виконання програми

Для отримання детальної статистики виконання базової версії програми використано команду:

/usr/bin/time --verbose ./bubble_base

Основні результати вимірювання:

- User time: 6.74 - процесорний час, витрачений на виконання коду програми в режимі користувача. Основну частину цього часу займають порівняння та перестановки елементів під час сортування.
- System time: 0.00 - час виконання операцій у режимі ядра Linux. Його незначне значення свідчить про те, що програма майже не потребує системних викликів під час основних обчислень.

- Percent of CPU this job got: 99% - програма майже весь час використовувала одне логічне ядро процесора. Це характерно для однопотокового обчислювального навантаження.
- Elapsed time: 0:06.75 - загальний час від запуску програми до її завершення. Він практично збігається з процесорним часом у режимі користувача.
- Maximum resident set size: 4048 - максимальний обсяг фізичної пам'яті, зайнятий процесом, становив приблизно 4,2 МБ.
- Major page faults: 0 - під час виконання не зафіксовано помилок сторінок, які потребували завантаження даних із диска.
- Minor page faults: 180 - кількість помилок сторінок, які оброблялися без дискового введення-виведення, наприклад під час підготовки пам'яті для процесу.
- Voluntary context switches: 2 - кількість добровільних перемикань контексту процесу.
- Involuntary context switches: 18 - кількість примусових перемикань контексту, під час яких планувальник операційної системи передавав процесорний час іншим завданням.
- Page size (bytes): 4096 - розмір одного блоку пам’яті, з яким операційна система працює під час керування віртуальною пам’яттю
- Exit status: 0 - програма завершилася успішно.

Інші показники, зокрема середні розміри спільної пам'яті, стека та кількість мережевих повідомлень, дорівнювали нулю. У цьому запуску вони не мають істотного значення для аналізу алгоритму сортування.

Аналіз продуктивності програми за допомогою perf stat

Для збору статистики апаратних і програмних подій використовувалася команда:

```
perf stat -e
task-clock,context-switches,cpu-migrations,page-faults,cycles,instructions
./bubble_base
```

Основні результати вимірювання:

- task-clock - сумарний процесорний час, протягом якого виконувалася програма, становив приблизно 7027,71 секунди. Інструмент також показав 0.999 CPUs utilized, тобто програма використовувала майже один логічний процесор протягом виконання.
- context-switches - зафіксовано 0 перемикань контексту для досліджуваного процесу.
- cpu-migrations - значення 0 означає, що інструмент не зафіксував переміщень виконання процесу між процесорами.
- page-faults - зафіксовано 161 помилку сторінки пам’яті. Саме по собі це значення не свідчить про проблеми з продуктивністю, оскільки такі події можуть виникати під час звичайного виділення та підготовки пам’яті.
- cycles - значення не підтримується в поточному середовищі, тому кількість тактів процесора визначити не вдалося.
- instructions - значення не підтримується, тому кількість виконаних процесорних інструкцій також не вдалося визначити.
- time elapsed - реальний час виконання програми становив приблизно 7,04 секунди.
- user - процесор витратив приблизно 7,02 секунди на виконання коду програми в режимі користувача.
- sys - на виконання операцій у режимі ядра витрачено приблизно 0,008 секунди.

Аналіз виконання програми за допомогою perf report

Для аналізу виконання програми спочатку було записано дані за допомогою perf record, після чого їх проаналізовано в perf report.

Команди для запису профілювання обох версій:

```
sudo perf record -F 99 -g -o perf_base.data -- ./bubble_base

sudo perf record -F 99 -g -o perf_opt.data -- ./bubble_opt
```

Для перегляду результатів використовувалися команди:

```
sudo perf report -i perf_base.data
```

sudo perf report -i perf_opt.data

Параметр -F 99 задає частоту семплювання приблизно 99 разів на секунду, -g вмикає запис стеків викликів, а -o задає окремий файл для результатів кожної версії.

У профілі базової версії було отримано такі результати:

- bubble_sort - приблизно 95,82% за показником Children, що підтверджує домінування сортування в загальному процесорному навантаженні.
- bubble_sort - приблизно 25,40% за показником Self, тобто така частка семплів припадала безпосередньо на код цієї функції, без урахування вкладених викликів.
- std::vector<int>::operator[] - приблизно 50,36% за Self, що вказує на значну частку семплів, пов'язаних із доступом до елементів вектора.
- std::vector<int>::size() - приблизно 20,20% за Self, що відображає витрати, пов'язані з отриманням розміру вектора.
- std::swap і std::move - приблизно 4,04% та 1,01% відповідно.

Показник Children враховує час функції разом із вкладеними викликами. Показник Self характеризує частку семплів, безпосередньо віднесених до самої функції. Тому ці показники не потрібно додавати між собою.

Під час порівняння двох версій кількість зафіксованих семплів зменшилася з 767 у базовій версії до 264 в оптимізованій. У профілі оптимізованої версії виклики operator[] і size() більше не відображалися як окремі значущі функції, а показник Self для bubble_sort становив 100%.

Це узгоджується з тим, що оптимізований компілятором код може виконувати доступ до елементів вектора без окремих викликів функцій, наприклад завдяки інлайнінгу та спрощенню операцій із пам'яттю.

Водночас кількість семплів залежить від тривалості запуску та налаштувань профілювання. Саме зменшення кількості семплів не є точним вимірюванням прискорення, але разом із результатами time і perf stat воно підтверджує скорочення обчислювальних витрат.

Висновок. Профіль perf report показує, що основним джерелом навантаження базової версії є bubble_sort, а значна частка семплів

пов'язана з операціями доступу до вектора. В оптимізованій версії ці операції значною мірою усуваються як окремі виклики, що узгоджується зі скороченням часу виконання.

Оцінювання енерговитрат програми

Для приблизного оцінювання енерговитрат було створено скрипт power_consumption.sh, який запускає програму, вимірює час її виконання та обчислює оцінки енергії на основі заданих значень потужності.

Перед запуском скрипта встановлюється утиліта bc, яка використовується для арифметичних обчислень:

sudo apt update && sudo apt install bc -y

Скрипт запускається командою:

./power_consumption.sh ./bubble_base

./power_consumption.sh ./bubble_opt

У скрипті задано такі параметри:

- P_IDLE = 5 - умовна потужність системи в стані простою, у ватах.
- P_DYNAMIC = 25 - умовна додаткова потужність під час виконання обчислень, у ватах.

Для вимірювання часу використовується /usr/bin/time. Отримані значення загального часу, часу користувача та системного часу застосовуються для оцінювання енергоспоживання за формулою (E=P\cdot t).

Скрипт обчислює два показники. Total System Energy оцінює загальні енерговитрати за заданою моделлю, а Program-Specific Energy оцінює додаткові енерговитрати, пов'язані з виконанням програми.

Для оцінювання впливу оптимізації на енергоефективність було порівняно результати вимірювань базової та оптимізованої версій програми за обома показниками.

Після оптимізації програми енерговитрати суттєво зменшилися. Загальне енергоспоживання системи (Total System Energy) скоротилося з 380,60 Дж до 131,45 Дж, а оцінене енергоспоживання програми (Program-Specific Energy) — з 346,00 Дж до 119,50 Дж. У першому випадку зменшення становить приблизно 65,5%, у другому — 65,5%.

Основною причиною такого результату є скорочення часу виконання програми з 6,92 до 2,39 секунди, тобто приблизно у 2,9 раза. Водночас середня потужність системи не змінилася й становила 55,00 Вт для обох версій. Отже, оптимізація насамперед зменшила тривалість обчислень, а не потужність, яку система споживала в одиницю часу. Завдяки швидшому виконанню того самого завдання загальні енерговитрати суттєво скоротилися.

Зменшення часу виконання можна пояснити оптимізацією машинного коду: усуненням зайвих викликів функцій, ефективнішим доступом до елементів масиву та використанням SIMD-інструкцій AVX під час перевірки відсортованості масиву. Завдяки цьому процесор виконує необхідну роботу швидше, а система витрачає менше енергії за весь час роботи програми.

Таким чином, оптимізована версія є енергоефективнішою, оскільки виконує те саме завдання за значно коротший час і з меншими сумарними енерговитратами.

Пояснення різниці в асемблерному коді до та після оптимізації

Порівняння асемблерного коду базової та оптимізованої версій програми сортування бульбашкою показує, що оптимізація компілятора змінює спосіб виконання циклів, доступу до елементів вектора та перевірки відсортованості масиву.

1. Виклики функцій і вбудовування коду (Inlining)

У базовій версії функції bubble_sort присутні окремі виклики методів std::vector:

call "std::vector<int, std::allocator<int>>::size() const"

call "std::vector<int, std::allocator<int>>::operator[](unsigned long)"

call "std::vector<int, std::allocator<int>>::operator[](unsigned long)"

Такі виклики створюють додаткові витрати на передавання аргументів, перехід до функції та повернення з неї. Зокрема, під час сортування оператор operator[] викликається для доступу до сусідніх елементів вектора, а size() - для отримання його розміру.

В оптимізованій версії ці операції переважно усунені шляхом вбудовування функцій (Inlining). Замість багаторазових викликів методів компілятор використовує безпосередні операції з адресами пам'яті та значеннями елементів. Наприклад:

mov r8, QWORD PTR [rdi]

mov rdi, QWORD PTR [rdi+8]

sub rdi, r8

sar rdi, 2

Ці інструкції отримують адреси початку й кінця області даних вектора, обчислюють різницю між ними та ділять її на розмір одного елемента (4 байти). Таким чином, компілятор може визначити кількість елементів без окремого виклику size().

2. Оптимізація циклів сортування

У базовій версії внутрішній цикл містить численні операції з індексами, завантаження адрес вектора та виклики operator[]. Наприклад:

mov eax, DWORD PTR [rbp-24]

movsxd rdx, eax

mov rax, QWORD PTR [rbp-40]

mov rsi, rdx

mov rdi, rax

call "std::vector<int, std::allocator<int>>::operator[](unsigned long)"

Після виклику функції значення елемента завантажується з пам'яті. Подібна послідовність повторюється для сусіднього елемента та під час обміну елементів.

В оптимізованій версії доступ до елементів виконується безпосередньо:

```
mov edx, DWORD PTR [rax]

mov ecx, DWORD PTR [rax+4]

cmp edx, ecx

jle .L14

mov DWORD PTR [rax], ecx

mov DWORD PTR [rax+4], edx
```

Тут rax містить адресу поточного елемента, а rax+4 - адресу наступного. Інструкція cmp порівнює значення, jle переходить до наступної ітерації, якщо обмін не потрібен, а дві інструкції mov виконують обмін значень у пам'яті.

Отже, компілятор усунув проміжні виклики функцій і реалізував порівняння та обмін елементів безпосередньо в машинних інструкціях.

3. Оптимізація перевірки відсортованості

У базовій версії функція check_if_sorted використовує цикл, у якому повторно викликаються operator[] та size(). Це збільшує кількість операцій, необхідних для перевірки сусідніх елементів.

В оптимізованій версії компілятор перетворив цю логіку на послідовність прямих звернень до пам'яті, перевірок умов і векторних операцій. Це дає змогу обробляти кілька елементів за одну операцію, коли процесор підтримує відповідні інструкції AVX.

4. Скорочення накладних витрат

У базовому коді присутні додаткові інструкції для роботи зі стеком, збереження регістрів, передавання аргументів і виклику функцій.

Наприклад, push, pop, call та leave використовуються для організації викликів і повернення з функцій.

В оптимізованому коді частина таких операцій зникає або скорочується, а дані частіше обробляються безпосередньо в регістрах процесора. Це зменшує кількість інструкцій і накладні витрати на виконання алгоритму.

Водночас оптимізація не змінює принцип роботи сортування бульбашкою: елементи й надалі порівнюються попарно та обмінюються місцями, якщо вони розташовані в неправильному порядку.

Використання AVX-розширень та SIMD-інструкцій

В оптимізованому асемблерному коді функції check_if_sorted присутні інструкції AVX, які дають змогу застосовувати технологію SIMD (Single Instruction, Multiple Data) - виконувати одну операцію над кількома елементами даних одночасно.

У базовій версії перевірка відсортованості виконується переважно послідовно: програма завантажує два сусідні елементи, порівнює їх і переходить до наступної пари. Це видно з інструкцій mov, cmp та jle.

В оптимізованій версії використовуються такі інструкції:

vmovdqa ymm0, YMMWORD PTR [rdi+rax*4]

vpcmpgtd ymm0, ymm0, YMMWORD PTR [rsi+rax*4]

vptest ymm0, ymm0

Їхнє призначення:

- vmovdqa - завантажує 256-бітний блок даних у регістр ymm0. Він може містити вісім 32-бітних цілих чисел.
- vpcmpgtd - паралельно порівнює вісім пар 32-бітних цілих чисел зі знаком і формує маску результатів порівняння.
- vptest - перевіряє маску, щоб визначити, чи є серед результатів порівняння ненульові біти. Якщо всі елементи задовольняють

потрібну умову, програма може перейти до обробки наступного блоку.

Таким чином, замість послідовного порівняння кожної пари елементів компілятор використовує векторні операції, які дають змогу перевіряти до восьми пар 32-бітних чисел за один прохід відповідної послідовності SIMD-інструкцій. Фактична кількість виконаних операцій і швидкість залежать від даних та переходів програми.

У коді також присутня інструкція vzeroupper, яка очищає старшу частину векторних регістрів YMM перед переходом до коду, де це може бути важливим для ефективності виконання.

Варто зазначити, що AVX-інструкції використовуються насамперед під час перевірки відсортованості масиву, а не в основному циклі сортування бульбашкою. У самому алгоритмі сортування оптимізація полягає передусім у прямому доступі до пам'яті, усуненні зайвих викликів функцій та скороченні кількості допоміжних операцій.

Порівняння статистики виконання основної та оптимізованої програми

Маємо наступні результати виконання команди /usr/bin/time --verbose для оптимізованої програми:

- User time: 2.39
- System time: 0.00.
- Percent of CPU this job got: 99%
- Elapsed time: 0:02.39
- Maximum resident set size: 4044
- Major page faults: 1
- Minor page faults: 179
- Voluntary context switches: 5
- Involuntary context switches: 5
- Page size (bytes): 4096
- Exit status: 0

Оптимізована програма виконалася за 2,39 секунди, використовуючи в середньому 99% одного процесорного ядра. Максимальне споживання фізичної пам’яті становило приблизно 4,0 МіБ. Порівняно з базовою версією, яка виконувалася за 6,75 секунди, час виконання зменшився

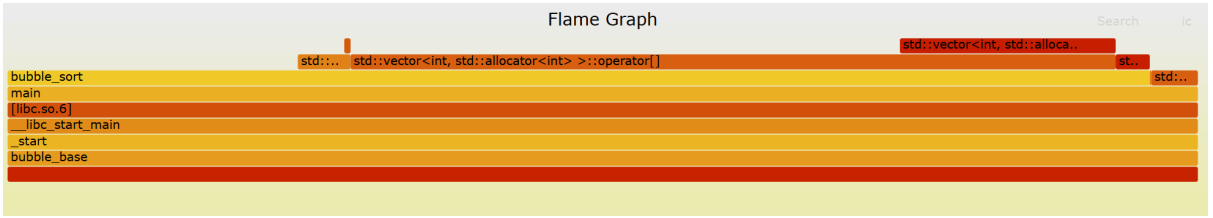
приблизно на 64,6%, тобто оптимізована програма працювала приблизно у 2,82 раза швидше. Це свідчить про суттєве покращення продуктивності після оптимізації.

Також маємо наступні результати виконання команди perf stat -e task-clock,context-switches,cpu-migrations,page-faults,cycles,instructions ./bubble_opt для оптимізованої програми:

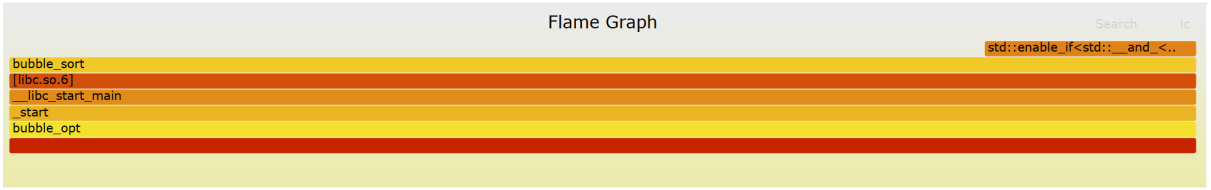
- task-clock - сумарний процесорний час, протягом якого виконувалася програма, становив приблизно 2372,52 мс (2,37 секунди). Інструмент також показав 1.000 CPUs utilized, тобто програма використовувала майже один логічний процесор протягом виконання.
- context-switches - зафіксовано 0 перемикань контексту для досліджуваного процесу.
- cpu-migrations - значення 0 означає, що інструмент не зафіксував переміщень виконання процесу між процесорами.
- page-faults - зафіксовано 162 помилки сторінки пам’яті.
- cycles - значення не підтримується в поточному середовищі.
- instructions - значення не підтримується у поточному середовищі
- time elapsed - реальний час виконання програми становив приблизно 2,37 секунди.
- user - процесор витратив приблизно 2,37 секунди на виконання коду програми в режимі користувача.
- sys - на виконання операцій у режимі ядра витрачено приблизно 0,004 секунди.

Після оптимізації реальний час виконання програми зменшився приблизно на 66,3% - з 7,04 до 2,37 секунди. Оптимізована версія працювала приблизно у 2,97 раза швидше. Обидві версії використовували майже один логічний процесор і не мали зафіксованих перемикань контексту чи міграцій між процесорами. Кількість помилок сторінок майже не змінилася, тому основне покращення продуктивності пов’язане не з істотним зменшенням цього показника.

Зміни на FlameGraph після оптимізації



FlameGraph до оптимізації



FlameGraph після оптимізації

Порівняння FlameGraph до та після оптимізації наочно демонструє структурні зміни в роботі програми після застосування оптимізацій (-O3 -march=native -flto):

1. **Зникнення накладних витрат стандартних контейнерів:**
    a. На графіку дооптимізації чітко видно широкі блоки std::vector<int, std::allocator<int> >::operator[], що свідчить про значні витрати процессорного часу на індексацію та перевірки безпеки доступу до елементів вектора всередині циклів сортування.
    b. На графіку оптимізованої версії ці блоки повністю відсутні, що підтверджує успішне усунення накладних витрат абстракцій C++.
2. **Сплощення стека викликів (Inlining):**
    a. Структура викликів оптимізованої стала значно простішою та плоскішою. Завдяки агресивному інлайнінгу (inlining) компілятор вбудував дрібні операції безпосередньо в тіло сортування, прибравши зайві переходи та виклики функцій.
3. **Концентрація часу на алгоритмі:**
    a. Увесь корисний час виконання оптимізованої програми зосереджений безпосередньо в оптимізованому коді сортування (bubble_sort), що робить виконання максимально ефективним і компактним.